

Introduction to Operating Systems

Luís Lopes

Faculdade de Ciências
Universidade do Porto

`lmlopes@fc.up.pt`

Operating Systems

- 1 definition, goals, examples
- 2 multiprogramming and multitasking
- 3 the UNIX system
- 4 kernel and system calls
- 5 system services
- 6 command shells
- 7 compilers and libraries
- 8 bootstrap process
- 9 interrupts

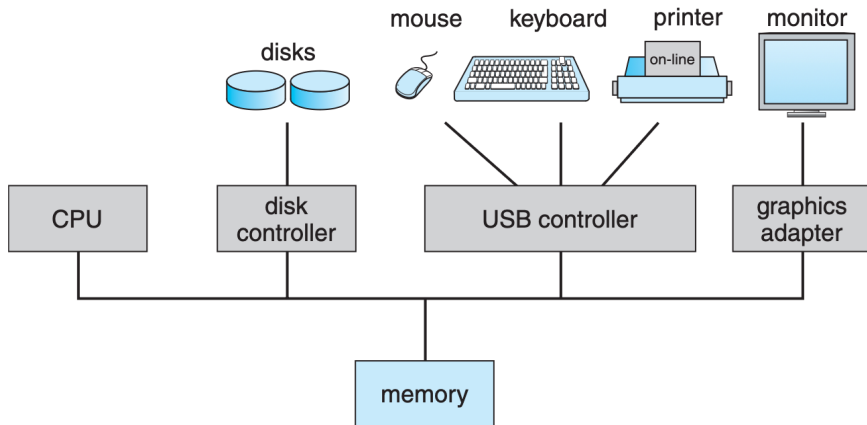
Operating Systems

- one possible definition of operating system:

"A software that is continuously running on a computer and that manages and provides the hardware resources required for the execution of user applications"

- its main goals are:
 - efficient access to hardware resources
 - fair access to hardware resources
 - integrity of hardware and information stored therein
 - note: hardware = CPU + memory + storage + I/O devices

Operating Systems



Operating Systems

- there are hundreds of operating systems
- most of them are used in particular contexts
- ..., e.g., control an industrial machine
- you already know some mainstream operating systems
- ..., e.g., Windows, Linux, macOS, Android, iOS
- Android descends from Linux and iOS from macOS
- Linux descends from UNIX (from ~ 1970)
- macOS descends from BSD UNIX
- Windows is an evolution of MSDOS (from ~ 1980)
- UNIX/Linux will be the main reference for this course

Operating Systems

operating systems typically match application contexts:

- supercomputers - batch execution, raw performance
- servers - multitasking, handle many clients simultaneously
- desktop/laptop - responsiveness, ease of use
- mobile - responsiveness, ease of use, preserve battery
- embedded devices - must run with low resources
- real-time - ensure applications meet deadlines

Multiprogramming vs. Multitasking

- operating system should support multiprogramming
- increases CPU utilization by organizing processes
- ... so that the CPU always has one process (p) to execute
- OS keeps several processes in memory simultaneously
- ... picks and begins to execute one of the processes in memory
- eventually, that process may have to wait, e.g., I/O operation
- in non-multiprogrammed OS, CPU would sit idle
- in multiprogrammed one, OS switches to another process (q)
- when q needs to wait, the CPU switches to another, and so on
- eventually, process p finishes waiting and gets the CPU back

Multiprogramming vs. Multitasking

- multiprogramming in itself is not designed for user interaction
- multitasking(= time-sharing) is extension of multiprogramming
- the CPU executes multiple jobs by switching among them
- ... the switches being so frequent that the users can interact
- ... with their processes while they are running

Multiprogramming vs. Multitasking

- multitasking is typically used in interactive computer systems
- ... providing direct communication between user and system
- ... e.g., user gives instructions to OS or to a process directly
- ... using device such as keyboard, mouse, touch-pad/screen
- ... and waits for immediate results on an output device
- response time should be short—typically less than one second

Multiprogramming vs. Multitasking

- thus in a multitasking OS:
- many users share the computer simultaneously
- the OS switches processes with high frequency, e.g., 10 s^{-1}
- each user has illusion that entire computer is dedicated to him
- *this is how servers, desktops, laptops, tablets and phones work*

The UNIX Operating System



The UNIX Operating System

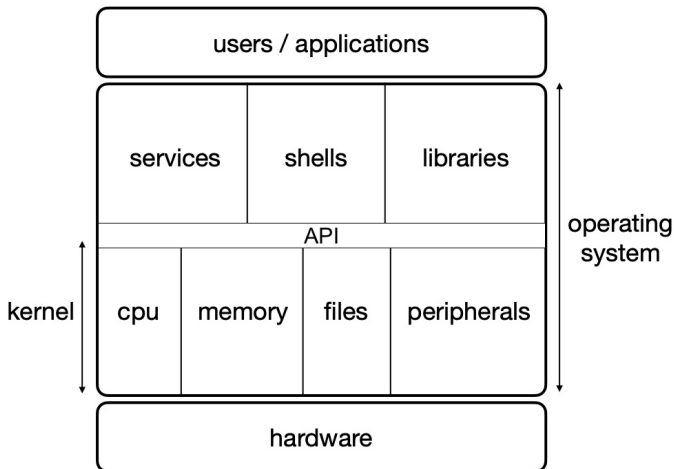
- UNIX (Bell Labs 1969, Dennis Ritchie and Ken Thompson)
- innovative operating system (and C programming language)
- implements multitasking
- adaptability - can be adjusted to work in different contexts
- reliability - less prone to errors or security issues
- portability - can run on different hardware architectures
- scalability - can be used in many different scales ...
- ... e.g., from networks, supercomputers down to Raspberry Pi

The UNIX Operating System

- implementing multitasking required innovation
- at the software level, when the OS switches processes ...
- ... the state of the current process must be saved ...
- ... the state of the next process must be loaded to the CPU
- this implies additional kernel data structures
- $\implies$ Process Control Block or PCB
- at the hardware level:
- *timers* to keep track of when to make a switch
- *interrupts* to tell CPU to stop and switch
- *interrupt masking* to avoid switching under certain conditions

The UNIX Operating System

- structure of the UNIX operating system



The UNIX Operating System

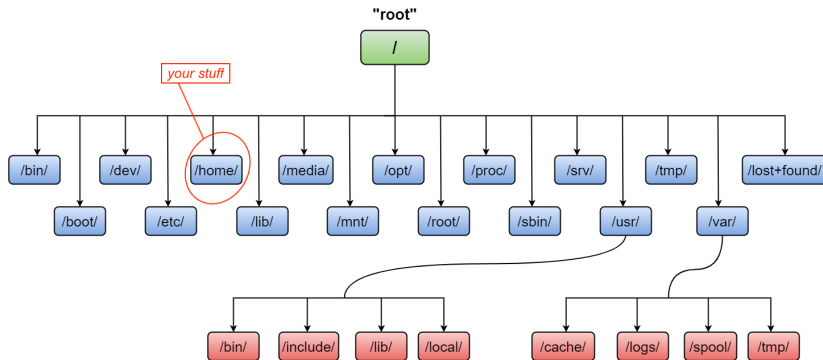
- the *kernel* is the part of the operating system ...
- ... that directly controls the hardware
- (kernel means "core, heart")
- in UNIX it is divided into 4 components:
- ... CPU management
- ... memory management
- ... file systems (storage management)
- ... I/O device management (peripherals)

The UNIX Operating System

- this type of organization is often called "monolithic"
- ... as all kernel functionality is in one piece
- other systems have smaller (micro-) kernels
- ... typically handling only CPU and memory management
- storage and peripheral management are implemented as
- ... operating systems services, outside the kernel
- there are pros and cons to both monolithic and micro designs
- in general monolithic kernels are faster but bigger
- ... lack modularity and more prone to security issues
- in general micro-kernels are smaller, slower,
- ... modular and easier to secure

The UNIX Operating System

- typical structure of file-system in Linux



The UNIX Operating System

- besides the kernel, the OS provides compilers and libraries
- ... typically C-based, to help in application development
- libraries are archives containing pre-compiled code
- ... for often-requested functions, e.g., math, I/O, graphics
- libraries come in two flavors: static and dynamic
- static libraries are used at compile time to construct a `.out`
- code from dynamic libraries is invoked at runtime
- they are shared by different processes and
- ... kept in memory by the operating system

The UNIX Operating System

- the OS also provides command interpreters,
- ... usually called *command shells*, e.g., `sh`, `csh`, `bash`
- these provide an alternative way of interacting with the kernel
- a set of predefined commands (binary executable files
- ... usually in `/bin`, `/usr/bin` or `/usr/sbin`)
- ... is available and allows users
- ... to perform operations that involve the kernel
- ... mostly for file management, process management and
- ... inter-process communication
- (c.f., exercise sheet #0 in practical classes)

The UNIX Operating System

- finally, OS provides multiple built-in services
- they perform maintenance and manage resources
- these services typically run as *daemons*:

“something not visible but always present and working its will”

- e.g.,
- `swapper` - transfer to/from memory from/to disk
- `init` - process #1 in UNIX, creates many others
- `inetd` - manages network access
- `dhcp` - get local IP address
- `ftpd` - file transfer
- `smtpd` - email service
- `sshd` - secure shell
- `portmap` - manages process connections

The UNIX Operating System

- in UNIX *"everything is a file"*, e.g.,
- files (obviously) - to store data
- directories(=folders) - to organize files
- pipes, sockets, fifos - for inter-process communication
- devices - e.g., `/dev/sda1` is partition 1 of first disk
- memory - e.g., memory mapped files with `mmap`
- state - e.g., process information in `/proc`
- all these obey to the same open-read/write-close
- ... semantics used with common files
- this abstraction simplifies the design of the system and APIs

System Bootstrap

- when we turn on a computer, the operating system
- ... namely its kernel, is stored in disk
- ... must be copied to memory and executed
- ... before the computer is usable

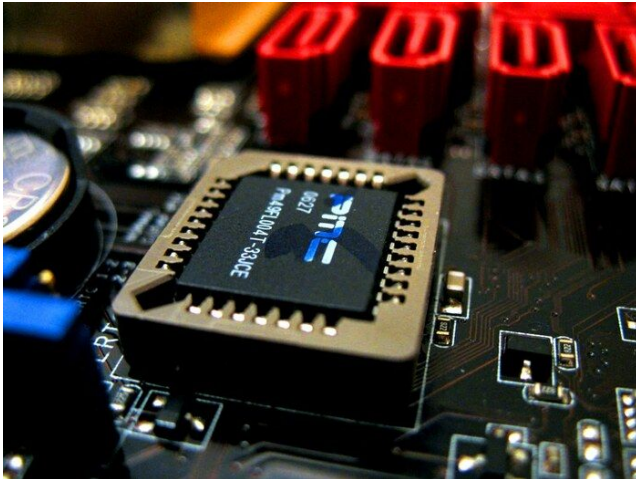
- *bootstrap* is the name given to an operation that
- ... loads a large software component into memory
- ... with the help of a small auxiliar program
- in the context of turning on the computer:
- large software component = OS kernel
- small auxiliar program = BIOS firmware (bootloader)

System Bootstrap

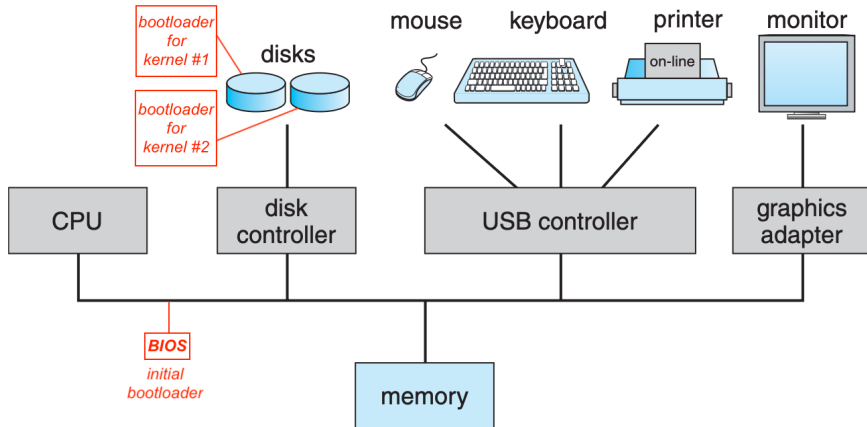
- BIOS = Basic Input/Output System
- *firmware* is the name given to low-level software
- ... provided together with hardware components
- ... that contains programs, like the bootloader
- ... and stores system parameters
- ... e.g., CPU clock speed, boot device
- the BIOS is embedded in the motherboard
- BIOS come in two flavors:
- older 16-bit BIOS with small ROM/EPROM memories
- newer (≥ 2006) 64-bit UEFI BIOS with large (fast) NVRAM
- UEFI = Universal Extensible Firmware Interface
- most new computers have UEFI BIOS

System Bootstrap

- a BIOS integrated circuit in the motherboard



System Bootstrap



System Bootstrap

- the bootloader (B) is stored in the ROM/EPROM or NVRAM
- when the computer is turned on, B is executed
- ... performs a basic hardware check
- ... identifies OS kernels in disks and selects one
- ... transfers control to kernel-specific bootloader (B_k) in disk
- ... B_k runs and transfers the kernel to memory
- ... the kernel is executed
- ... the system services are initiated (`init`)
- ... the login prompt is presented
- ... the system is ready

Event Driven Operating System

- ok, the system is ready ... now what?
- operating systems like UNIX or Windows
- ... are driven by hardware and software events

Event Driven Operating System

- *hardware-triggered* events are called *interrupts*
- a hardware component puts some voltage
- ... on a motherboard line connected to the CPU
- e.g.,
- ... user hits the keyboard
- ... user moves the mouse
- ... an external disk is inserted
- ... data arrives at the network card
- ... internal timer
- ... memory corrupted
- ... power outage
- interrupts are asynchronous (can happen anytime)

Event Driven Operating System

- *software-triggered* events are called *traps*
- a process executes an invalid operation
- or a problem occurs executing a valid one
- or a system call is required
- e.g.,
- ... division by zero (floating point exception)
- ... sqrt negative number (floating point exception)
- ... access to invalid address (segmentation fault)
- ... data missing in memory (page fault)
- ... execution of kernel code (system call)
- traps are synchronous (their occurrence predictable)
- implemented in hardware using interrupts!

Event Driven Operating System

- interrupts have an associated integer identifier (id)
- it indicates the type of event that occurred
- the kernel has a *dispatch table*
- for each interrupt id, the table keeps a pointer
- ... to the function that should be called in response
- this function is called a *interrupt handler*, or just *handler*
- for many interrupts and traps
- ... the default action is to kill the current process
- e.g., segmentation fault, floating-point exception

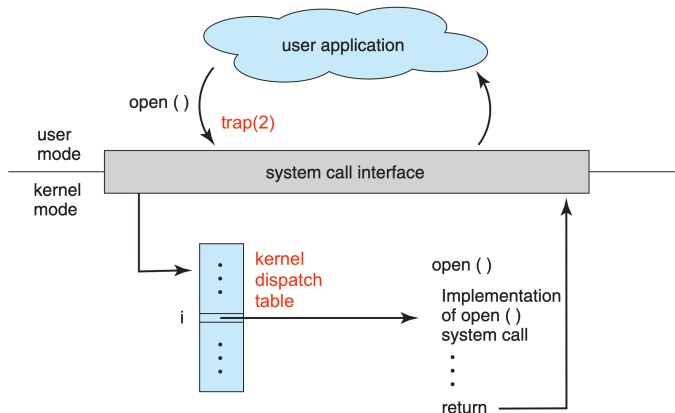
Event Driven Operating System

- assume the CPU is running a process p
- the interrupt with identifier ID handling cycle is as follows:
- ... interrupt is triggered ($\implies$ *raise*)
- ... CPU detects interrupt, stops p ($\implies$ *catch*)
- ... CPU finds handler for ID and executes it ($\implies$ *dispatch*)
- ... CPU clears ID and resumes process p ($\implies$ *clear*)

Event Driven Operating System

- system calls = calls to functions whose code is part of kernel
- they are implemented using traps
- e.g.,
- ... system call `open()`
- ... the entry point of the function is in `libc`
- ... the function in `libc` executes `trap(2)` (Linux 64bit)
- ... the trap is processed as described in the previous slide
- ... handler for trap #2 has the kernel code for `open()`
- ... the handler returns
- ... `libc` function `open()` returns with file descriptor

Event Driven Operating System



Event Driven Operating System

- e.g.,
- executing commands from a shell triggers a system call
- the `exec()` system call transforms a command into a process
- with an address space and a Process Control Block (PCB)
- you write `$. /a.out`
- ... shell calls `exec()` with `./a.out` as argument
- ... the entry point of the function is in `libc`
- ... the function executes `trap(59)` (Linux 64bit)
- ... the trap is processed as described above
- ... *handler for trap #59 has the kernel code for `exec()`*
- ... the handler returns
- ... `libc` function `exec()` returns only if error is found
- your shell command is now a new process